

DEMO 08 • ADAPTIVE TUTORING

A Tutor That Reads the Room

A learner-state model that recommends a hint and easier work for a struggling student — and harder work for a strong one — as advice to the platform, never an imposed action.

ADVISORY Recommends only — no autonomous writes

AUTHOR Rakovskyi Dmytro • VERSION Full-run demo 08 • DATE 2026 • REPOSITORY github.com/rakovpublic/jneopallium

The difference between a good tutor and a bad one is rarely the material; it is timing. Push a frustrated student harder and they quit; coast a strong one and they switch off. This demo watches the signals a human tutor reads — whether answers are right, how long they take, whether hints get asked for — and infers mastery, frustration, and engagement, then recommends the next move. It advises the learning platform; it never seizes control of the lesson.

What it is

Demo 08 models adaptive tutoring over an LTI/xAPI-style stream of learner events. Its safety ceiling is **ADVISORY**: it recommends a difficulty change or a hint, and the platform — and ultimately the educator — decides. The network is four layers (sized 4·3·2·2), enough to turn raw learning events into an affect-and-mastery estimate and a recommendation.

How it is wired

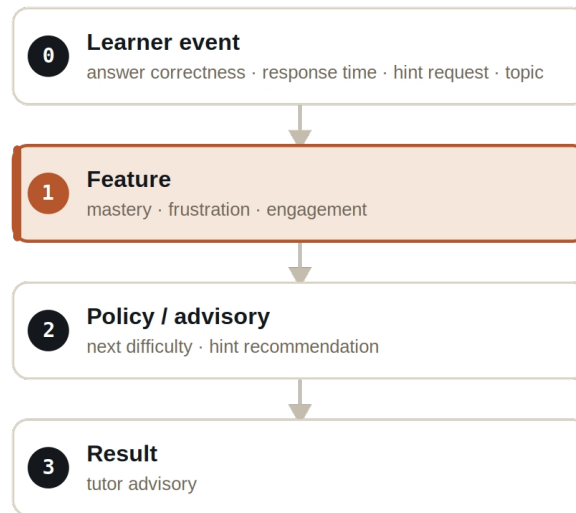


Figure Four layers. A feature layer estimates the learner's state; a policy layer turns it into a recommendation.

Signals are typed: `LearnerEventSignal` carries the raw events, `MasterySignal` carries the competence estimate, `AffectSignal` carries frustration and engagement, and `TutorAdvisorySignal` carries the recommendation.

What it does

Two synthetic learners run through the same model, and the recommendations diverge exactly as a good tutor's would.

- **A struggling learner** — repeated wrong answers and long response times — earns a recommendation to offer a hint and lower the difficulty.
- **A strong learner** — fast, correct answers — earns a recommendation to raise the difficulty with a harder exercise.
- **Both outputs are advisory:** the platform applies, defers, or overrides them, and an educator stays in charge of the pedagogy.

NOTE

Same model, opposite advice. The point of the demo is that the recommendation tracks the learner's state rather than applying a one-size schedule.

The evidence

VERIFICATION EVIDENCE

Ticks

100 / 100

Struggling learner	RECOMMEND_HINT_AND_LOWER_DIFFICULTY
Strong learner	RECOMMEND_HARDER_EXERCISE
Imposed actions	none — advisory only
Verdict	PASS

How to run it

1 Build

Build the worker and the demo model JAR.

```
mvn -q -pl worker -am package
```

2 Run via the framework entry point

Entry loads the model JAR and the demo context through the real local path.

```
java -cp demo-model.jar \  
  com.rakovpublic.jneuropallium.worker.application.Entry \  
  local file:///path/to/demo-model.jar \  
  com.rakovpublic.jneuropallium.worker.demo.fullrun.runtime.DemoJsonContext \  
  context.json
```

3 Inspect

Read the tutor advisories for both learners.

```
ls target/jneopallium-fullrun-demos/demo-08-tutoring/
```

Why ADVISORY

Pedagogy is a human responsibility and learners are not optimisation targets; a system that silently re-sequences a curriculum has overstepped. The advisory ceiling keeps the educator and platform in control of what actually happens. To run it in a live course you replace the mock event edge with a real LTI/xAPI source while preserving the typed signal contract and the advisory ceiling — the recommendation arrives the same way, and a human still decides how to teach.